

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect	3

	classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect	3



Engineer to Excel



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



patterns can be identified. Suggest improvements to prevent recurring defects.

13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves	3



	quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	
--	--	--



Code of Conduct:

I KARTHIGA G(192421383) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

KARTHIGA G

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1 | Banking App: Replacing Ad-hoc Testing

Scenario

A banking application team faces frequent failures because testing is unstructured. As a test engineer, redesign the approach using testing as an engineering activity.

1. Core idea: testing as an engineering activity

Testing is treated like any engineering discipline: it is **planned, designed, measured and repeatable**. Instead of "click around and see", each test has a purpose, input, expected result and a record of what happened.

2. Ad-hoc testing versus engineered testing

Aspect	Current (ad-hoc)	Redesigned (engineering approach)
Planning	None; testers pick screens at random	Test plan with scope, risks, schedule, entry/exit criteria
Test design	Made up during execution	Designed from requirements using EP, BVA, decision tables
Tools	Manual, notes in chat	Test management (TestRail/Jira), automation (Selenium), API tests (Postman)
Standards	None	IEEE 829 documents, ISO/IEC 25010 quality attributes, bug template
Measurement	No data	Requirement coverage, defect density, pass rate, defect leakage
Repeatability	Cannot repeat a failed test	Every test case is documented and reusable for regression

3. How to introduce systematic design, tools and standards



- **Systematic design:** derive tests from each requirement, and keep a traceability matrix (requirement to test case to defect).
- **Tools:** test management tool for cases and runs, automation for regression of login and transfers, a defect tracker.
- **Standards:** IEEE 829 test documentation, a common defect template, and a review checkpoint before execution.

4. How this improves defect detection and quality

- Requirement coverage is visible, so untested features are found before release.
- Techniques target the places where defects hide (boundaries, invalid states), so more defects are found per test.
- Regression suites stop old defects returning, and metrics let the team see quality trends.

5. Techniques that replace ad-hoc testing

Technique	Use in banking app
Equivalence partitioning	Transfer amounts split into valid (1 to limit), zero/negative, above limit

Technique	Use in banking app
Boundary value analysis	Test 0, 1, limit, limit+1 for the daily transfer limit
Decision table	Combinations of balance, account status and limit for approving a transfer
State transition	Account states: active, locked after 3 wrong PINs, unlocked
Error guessing + exploratory (time-boxed)	Session timeout during payment, double-click on "Pay"

6. Practical example

Fund transfer with a daily limit of Rs. 50,000

Ad-hoc: tester tries Rs. 1,000 and Rs. 20,000, both pass, so the feature is declared fine.

Engineered: BVA gives 0, 1, 49,999, 50,000, 50,001. The test for 50,001 shows the system accepts it. This is a defect that ad-hoc testing missed, and it is logged with steps, data and severity.

Conclusion: structured design, proper tools and standards turn testing into a measurable engineering process, which reduces failures in the banking application.

Question 2 | ATM System: Structured Testing Across SDLC

Scenario

An ATM system gives inconsistent results because there is no defined testing process. Implement structured testing across SDLC phases and apply it to withdrawal.

1. The approach: testing in every SDLC phase (V-model view)



2. Stage, activity, relevance and ATM withdrawal application

SDLC phase	Testing activity	Why it matters for quality	Applied to ATM withdrawal
Requirements	Review requirements; write acceptance criteria; plan tests	Ambiguity found here costs least to fix	Clarify: limit per transaction, multiples of Rs. 100, card retention rules
Design	Review design; design system and integration test cases	Catches design flaws before coding	Check flow ATM to bank server to cash dispenser; define failure handling when server times out
Coding	Unit tests, code review, static analysis	Defects removed at the source	Unit-test balance check, limit check, denomination calculation
Testing	Integration, system, security, performance, acceptance tests	Validates the whole against requirements	End-to-end: insert card, PIN, withdraw, balance updated, receipt printed
Deployment	Smoke test, installation check	Confirms the release works in the real environment	Check cash cassette sensors and network link on a live ATM
Maintenance	Regression and defect-fix verification	Changes do not break working features	After a fee change, re-run all withdrawal tests

3. Sample test cases for ATM withdrawal

ID	Input	Expected result
W1	Valid card, correct PIN, amount within balance and limit	Cash dispensed, balance reduced, receipt printed
W2	Amount greater than balance	Request refused with message; balance unchanged
W3	Amount not a multiple of Rs. 100	Rejected with a clear message
W4	Wrong PIN three times	Card blocked/retained
W5	Network failure after debit but before dispense	Transaction reversed automatically

4. How process consistency reduces defects

- The same documented tests run every release, so results do not depend on who tests.
- Early reviews remove requirement and design defects before they become code.

-
- Entry and exit criteria stop untested builds from moving forward.
 - Defect data per phase shows where defects enter and lets the team fix the process.

5. Expected outcomes

Consistent withdrawal behaviour across all ATMs and releases.

Fewer production failures and lower rework cost.

Traceable requirements to tests to defects, with measurable quality.

Higher customer trust and fewer cash-reconciliation disputes.

Question 3 | Web Application: Errors, Defects and Failures

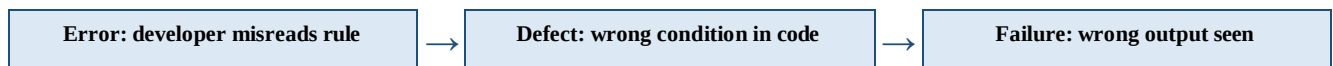
Scenario

A web app shows a mismatch between expected and actual output during login and transaction operations. Apply the basic definitions.

1. Definitions

Term	Meaning	Who/where
Error (mistake)	A human action that produces an incorrect result	Developer, analyst or designer
Defect (bug/fault)	An imperfection in the code or document caused by an error	In the software artefact
Failure	The system behaves differently from expected when the defect is executed	Seen at run time by the user/tester

2. Relationship: error leads to defect leads to failure



3. Applied to the login and transaction scenario

Scenario	Error	Defect	Failure
Login	Developer thinks usernames are case-sensitive but the spec says not	Code compares strings without lower-casing	Valid user "Ravi" cannot log in as "ravi"; expected: login success, actual: invalid credentials
Transaction	Developer rounds amounts with integer division	Decimal part dropped in calculation	Rs. 100.75 transfer shows Rs. 100 debited; expected Rs. 100.75

Note: a defect does not always cause a failure (for example, code that is never executed with the failing data). Failures can also come from the environment, not only from code defects.

4. How poor understanding of these terms hurts testing

- Testers report "login is broken" (failure) without the cause, so developers waste time reproducing.
- Teams count failures but not defects, so one defect behind five failures looks like five problems.
- Fixing symptoms instead of the defect makes the failure reappear.
- Metrics such as defect density become unreliable.

5. Improving defect reporting clarity

Weak report	Clear report
"Login not working"	Title: Valid username in lower case rejected at login Steps: 1. Open login page. 2. Enter "ravi" and valid password. 3. Click Login. Expected: Dashboard opens. Actual: "Invalid credentials". Environment: Chrome 126, build 2.3.1. Severity: High. Evidence: screenshot, log.

Use one standard template with unique ID, steps, expected versus actual result, environment, severity/priority and evidence. Separate the observed failure from the suspected defect.

Question 4 | Defects Concentrated in a Few Modules

Scenario

Most defects are in a few modules, causing delays. Apply the software testing principles to fix the planning.

1. Principles and how each guides test planning

Principle	Meaning	Planning action
Testing shows presence of defects	Testing cannot prove there are none	Report residual risk honestly; do not promise "bug-free"
Exhaustive testing is impossible	Cannot test all inputs and paths	Select tests by risk and technique, not by volume
Early testing	Start in requirements/design	Review requirements in the problem modules first
Defect clustering	A small number of modules hold most defects (Pareto 80/20)	Allocate more testers, time and depth to the high-defect modules
Pesticide paradox	Same tests stop finding new defects	Refresh and add new test cases each cycle
Testing is context dependent	Different systems need different testing	Use security-heavy tests for payment modules
Absence-of-errors fallacy	A defect-free but wrong product is useless	Include validation against user needs

2. Prioritising test cases

Score each module by **likelihood of failure** (past defects, complexity, code churn) and **business impact**, then test the high-high cell first.

	Low impact	High impact
High likelihood	Priority 2: test thoroughly but after P1	Priority 1: test first and deepest, automate regression
Low likelihood	Priority 4: smoke/sanity only	Priority 3: key scenarios plus monitoring

3. Real-time example

E-commerce release: 120 defects found in the last cycle

Defect data: Payment 48, Cart 31, Search 14, Profile 12, Others 15. Payment and Cart together hold 66% of defects.

Action: 60% of testing effort goes to Payment and Cart (boundary, negative, concurrency, security tests); Search and Profile get smoke and sanity tests.

Result: next cycle, the defect rate in Payment and Cart drops and the release date holds.

4. How this improves testing efficiency

- More defects found per hour of testing, because effort goes where defects live.
- Developers get early feedback on weak modules and can refactor or review them.
- Delays shrink because the riskiest areas are stable first.

-
- Remember to keep a light check on "quiet" modules, since clusters can move.

Question 5 | Unused Defect Repository and Repeated Defects

Scenario

A company keeps a defect repository but does not use it, so defects repeat across releases. Apply defect classification and repository concepts.

1. How defects originate



- Defects enter through any phase: unclear requirements, wrong design, coding slips, poor configuration, or missing tests.
- The root cause is usually a human error combined with a process weakness (no review, no checklist).

2. How defects can be categorised

Classification axis	Classes
By origin (phase)	Requirements, design, coding, testing, deployment
By type	Functional, logic, data, interface, performance, security, usability
By severity	Critical, major, minor, trivial
By priority	High, medium, low (business urgency)
By status	New, assigned, fixed, retested, closed, reopened

3. How repository data supports design and prevention

Repository data	Insight	Use
Defects per module	Hot spots	More tests and code reviews on those modules
Defects by type	Frequent logic errors in calculations	Add boundary and calculation tests; coding guideline
Defects by origin phase	Many from requirements	Introduce requirement review checklists
Reopened defects	Weak fixes	Require root-cause note before closing
Escaped defects	Missing test coverage	Create regression cases for each escaped defect

4. Improvements to avoid recurring defects

1. Perform root-cause analysis for every major defect and record it in the repository.
2. Convert each fixed defect into a regression test case.
3. Review defect trends at the start of each release planning meeting.
4. Keep mandatory fields (module, type, origin, severity, root cause) so data is analysable.
5. Share a "lessons learned" checklist with developers and testers.

5. Expected quality improvements

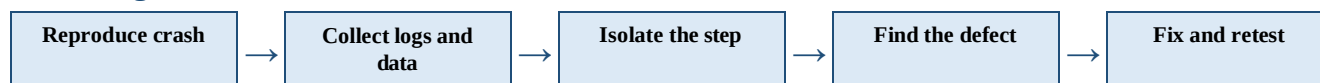
Fewer repeated and reopened defects.
Lower defect leakage to production.
Targeted test design and shorter test cycles.
Measurable defect density trend release over release.

Question 6 | Mobile Banking: Crash During Fund Transfer

Scenario

A mobile banking app often crashes during fund transfer. Identify the root cause with testing principles and prevent future failures.

1. Finding the root cause



Suspected cause	How it is tested	Principle applied
Unhandled null/empty field (amount, beneficiary)	Submit blank and special-character inputs	Testing shows presence of defects; negative testing
Large or boundary amount overflow	BVA: 0, 1, max limit, max+1	Defect clustering around boundaries
Network drop or timeout mid-transaction	Simulate 2G, airplane mode, server delay	Context-dependent testing (mobile)
Memory leak or low-memory device	Run on low-end devices; profile memory	Exhaustive testing impossible, so use risk-based device selection
Double tap on "Send" causing duplicate calls	Rapid repeated taps; concurrency tests	Early/negative scenario thinking
OS or app version incompatibility	Test matrix of Android/iOS versions	Coverage of environments

2. How systematic testing reduces such failures

- Every transfer rule is turned into test cases, including invalid and extreme data.
- Crash logs (Logcat, Crashlytics) are linked to test cases, so each crash becomes a permanent regression test.
- Automated regression runs on every build catch the crash before release.

3. Worked example

Crash when the amount field contains "1,000.50"

Failure: app closes on tapping Transfer.

Defect: code parses the amount with an integer parser, and the comma raises an exception that is not caught.

Error: developer assumed users type digits only.

Test added: inputs "1,000.50", "1000.505", "abc", "", "-5". **Fix:** validate input and handle exceptions with a friendly message.

4. Improvements to avoid future failures

Area	Improvement
Test design	Add negative, boundary, interruption (call, notification) and network tests
Devices	Cloud device farm covering top devices and OS versions by user share
Automation	Appium suite for transfer flow in the CI pipeline

Area	Improvement
Monitoring	Crash analytics in production and feedback to test cases
Process	Code review for exception handling; exit criteria need zero open crash defects

Question 7 | Testing versus Debugging

Scenario

The development team confuses debugging with testing, so defects are not resolved effectively. Differentiate the two and propose an integrated approach.

1. Testing versus debugging

Basis	Testing	Debugging
Purpose	Detect failures and show whether software meets requirements	Locate the cause of a failure and fix the defect
Performed by	Testers (and developers for unit tests)	Developers
Starts when	Test cases are executed	A failure has been reported
Output	Test results, defect reports	Corrected code
Planned?	Yes, scheduled and designed	Usually reactive
Needs code knowledge?	Not always (black box)	Always
Can be automated?	Largely yes	Mostly manual with tools

2. Their roles in software quality

- **Testing** measures quality and gives evidence of where the product fails.
- **Debugging** improves quality by removing the defects testing found.
- Quality needs both: testing without debugging only finds problems, and debugging without testing cannot confirm the fix.

3. What goes wrong when they are confused

Misuse	Effect on defect detection
Developers "test" only by debugging a known bug	New and unrelated defects are never discovered
Testers try to find the exact line of code	Time is lost; the tester role becomes unclear
Fix is not retested	Defect reappears or breaks another feature
Failures reported without steps	Debugging takes longer because the failure cannot be reproduced

4. A structured way to integrate both



- Use one defect tracker as the hand-off point between testers and developers.
- Rule: a defect is closed only after retest passes and regression shows no side-effects.
- Add a regression test for each fixed defect; hold short triage meetings to agree on severity.

Question 8 | E-commerce: Cross-Browser Testing Strategy

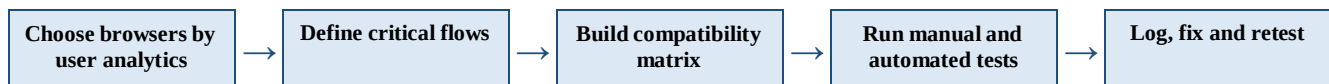
Scenario

An e-commerce platform behaves inconsistently across browsers. Design a cross-browser testing strategy.

1. Why defects originate in cross-browser situations

- Browsers use different rendering engines (Blink, Gecko, WebKit) and interpret CSS and JavaScript slightly differently.
- Developers write and test only in one browser; vendor-specific or new features are used without fallbacks.
- Differences in versions, screen sizes, operating systems, cookies and security settings add variation.

2. Strategy



Browser	Windows	macOS	Android	iOS
Chrome (latest, latest-1)	Full	Full	Full	Smoke
Firefox	Full	Smoke	Smoke	n/a
Safari	n/a	Full	n/a	Full
Edge	Full	Smoke	n/a	n/a

Full = all critical flows (search, product page, cart, checkout, payment, login). Smoke = main happy path only. Priorities come from real traffic data, since exhaustive testing of all combinations is impossible.

3. What to check

Category	Examples
Layout and UI	Alignment, fonts, images, responsive breakpoints
Functionality	Add to cart, coupon, form validation, payment redirect
Scripts	JavaScript errors in the console, AJAX calls
Compatibility	Cookies, local storage, pop-up and autofill behaviour
Performance	Load time per browser

4. Tools and techniques for better coverage

Need	Tool / technique
Real browsers and devices on cloud	BrowserStack, LambdaTest, Sauce Labs
Automated cross-browser scripts	Selenium Grid, Playwright, Cypress
Visual comparison	Percy, Applitools (screenshot diff)
Code-level compatibility	Can I Use checks, linters, polyfills/progressive enhancement
Prioritisation	Pairwise (combinatorial) testing to reduce combinations

How this improves reliability: the same test suite runs on every supported browser on each build, so inconsistencies are caught early and the user sees the same behaviour everywhere.

Question 9 | Payroll: Classifying Salary Calculation Defects

Scenario

A payroll system produces wrong salaries due to logical errors. Categorise the defects, explain why classification helps, and prevent recurrence.

1. Defect classification applied to the payroll system

Defect class	Payroll example	Severity
Requirements/specification	Overtime rule (1.5x after 8 hours) not clearly stated	Major
Algorithmic/logic	Tax slab applied to the whole salary instead of the portion above the slab limit	Critical
Control/sequence	Deductions applied before allowances, giving a wrong gross	Major
Data	Leap-year or unpaid-leave days read from the wrong table field	Major
Computation/precision	Rounding performed at every step instead of at the end	Minor to major
Interface	Attendance module sends days as text, payroll reads as integer	Major
Boundary	Employee earning exactly at slab limit taxed at the next rate	Major

2. How understanding defect classes improves testing

- It tells the tester which tests to design: logic defects need decision tables and BVA; data defects need data-flow tests.
- It shows where defects originate, so reviews can target the right phase.
- It makes defect statistics comparable and supports root-cause analysis.

3. Prevention in future releases

Defect class	Preventive action
Requirements	Formal review with finance/HR; examples for every pay rule
Algorithmic/logic	Peer code review, unit tests for each formula, decision-table tests for tax and overtime
Data and interface	Schema validation and integration tests between attendance and payroll
Precision	Use a decimal type, define rounding rules once, test with fixed expected values
Boundary	BVA on all slab limits and day counts

4. Worked example

Tax slab: 10% above Rs. 5,00,000

Salary Rs. 5,10,000. **Expected tax:** 10% of 10,000 = Rs. 1,000. **Actual:** 10% of 5,10,000 = Rs. 51,000. This is a logic defect (critical).

Tests added at 4,99,999, 5,00,000, 5,00,001 and 5,10,000 to catch it and similar slab errors.

5. Improvements in the testing strategy

- Parallel testing against a spreadsheet or trusted oracle with sample employees.
- Automated regression of payroll runs after every rule change; test month-end and year-end cases.
- Track defects by class so the most common class gets the most test effort.

Question 10 | Skipping Test Case Design: Random Tests

Scenario

A team skips test case design and runs random tests. Redesign the process using testing as an engineering activity.

1. Redesigned process



2. Importance of structured test case design

- **Coverage:** every requirement has at least one test, so nothing is forgotten.
- **Repeatability:** any tester can run the same test and get comparable results.
- **Efficiency:** fewer, better tests find more defects than many random ones.
- **Traceability and measurement:** test results can be reported against requirements.
- **Reuse:** cases become regression tests for later releases.

3. A standard test case template

Field	Example (login)
Test case ID	TC_LOGIN_03
Requirement ID	REQ-12 (account locks after 3 wrong attempts)
Precondition	Registered user, account active
Test steps	1. Enter valid user ID. 2. Enter wrong password three times.
Test data	User: ravi01; password: wrong123
Expected result	Account locked; message "Account locked, contact support"
Actual result / Status	To be filled during execution (Pass/Fail)

4. How this improves defect detection

Random testing repeats the same easy paths. Designed tests deliberately target boundaries, invalid input and combinations, which are where defects tend to be. Example: random testing may only try a valid login, while the designed set includes empty fields, locked account and maximum-length password and finds three extra defects.

5. Techniques to enhance effectiveness

If the input is...	Use
A range of values	Equivalence partitioning + boundary value analysis
Several conditions combining	Decision table / cause-effect
Different states over time	State transition testing
A user journey	Use case based testing
Unknown risk areas	Error guessing and time-boxed exploratory testing (as a supplement, with notes)

If the input is...	Use
Code structure to be covered	Statement and branch coverage (white box)

Question 11 | Healthcare: Data Inconsistency Across Modules

Scenario

A healthcare application shows data inconsistency across modules. Analyse the issue, explain defect origin, show how testing ensures data integrity, and improve defect tracking.

1. Analysis: where the same patient data travels



A change in one module (for example, a corrected date of birth or allergy) must appear in all others. Inconsistency means the data was copied, not shared, or an update was not propagated.

2. How such defects originate during development

Origin	Example
Requirements	No rule on which module is the "master" for patient data
Design	Duplicate tables for patient details in different modules
Coding	Update query changes the registration table but not the billing table
Integration	Date format DD/MM/YYYY in one module and MM/DD/YYYY in another
Data migration	Old records imported with missing fields

3. How systematic testing ensures data integrity

Test type	What is verified
Integration testing	Data passed between modules is complete and in the same format
End-to-end scenarios	Register patient, prescribe drug, dispense, bill: values match everywhere
Data validation	Field types, lengths, mandatory fields, formats
CRUD and consistency checks	Create, update, delete in one module reflected in others (SQL comparison queries)
Security and audit	Only authorised roles can change clinical data; audit trail recorded
Regression	Fixes do not break other module links

Example

Doctor updates allergy to "Penicillin". Test checks the pharmacy screen shows the alert and billing excludes it. If the pharmacy still shows "No allergy", the test fails and a high-severity defect is raised, since patient safety is affected.

4. Improvements in defect tracking and reporting

- Add fields: affected module(s), data entity (patient/prescription), severity with a "patient safety" flag, root cause and phase.
- Link each defect to the requirement and the test case that found it.
- Review open data-integrity defects in a weekly triage; report trends by module.
- Keep a master data definition so developers know the single source of truth.

Question 12 | Repeated Defects: Using the Defect Repository

Scenario

A company repeats defects in releases despite testing. Use the defect repository concept to analyse, support test design, find patterns and prevent recurrence.

1. What the defect repository is

A central, structured store of all defects with their attributes (ID, module, type, severity, origin phase, root cause, fix, status). It is the organisation's memory of past defects.

2. The analysis loop



3. How repository data supports test design

- High-defect modules and defect types tell testers where to design more or deeper tests.
- Each past defect becomes a regression test and an error-guessing checklist item.
- Origin data shows if reviews or requirement checks are missing.

4. Identifying patterns: worked example

Release	Total defects	Boundary defects	Repeated from earlier release	Reopened
R1	60	14	0	3
R2	68	20	12	7
R3	75	26	19	11

Pattern: boundary defects and repeats are rising each release, and reopened defects show weak fixes. **Action:** add boundary tests, create regression cases for all 19 repeats, require root-cause notes before closure.

5. Improvements to prevent recurrence

1. Enforce mandatory fields and consistent classification.
2. Run root-cause analysis (5 Whys / fishbone) on top defect types each release.
3. Add repeated defects to regression suite and review checklists.
4. Share monthly defect-trend dashboards with developers and managers.

6. Expected improvements

Repeated and reopened defect counts fall release over release.

Test effort is focused, so detection improves.

Fewer escapes to production and better customer satisfaction.

Question 13 | Learning System Failing at Peak Usage

Scenario

A web learning system fails during peak usage because scenarios were untested. Find the testing gaps and suggest improvements.

1. Gaps identified

Gap	Evidence	Missing test
No performance or load testing	Failure only when many users log in (exam day)	Load, stress and spike tests
Unrealistic test data and users	Tested with a few users and small data	Realistic data volume and user mix
No scalability check	Server and database limits unknown	Capacity and scalability tests
Incomplete test planning	Peak scenarios not in test plan	Risk-based plan including non-functional tests
No monitoring baseline	No response-time targets	Defined acceptance criteria (for example, 95% of pages under 3 s)

2. How test planning and coverage affect performance

- A plan with only functional tests gives coverage of features but zero coverage of load, so failures appear in production.
- Coverage should include **functional, non-functional, environment and data** dimensions.
- Defined targets (users, response time, error rate) make performance results measurable.

3. Better strategy: test types and examples

Test	Example for the learning system
Load test	5,000 concurrent students open lessons and submit quizzes
Stress test	Increase to 10,000 users to find the breaking point and recovery behaviour
Spike test	Sudden jump from 200 to 4,000 users when an exam opens
Endurance (soak) test	Run 8 hours to find memory leaks
Failover/recovery	Restart a server during a quiz and check no answers are lost

4. How this improves reliability



5. Enhancements to avoid such failures

- Use JMeter, Locust or k6 in the CI pipeline; test on a production-like environment.
- Analyse production traffic logs to build the load model; plan capacity (auto-scaling, caching, CDN).
- Monitor in production (APM tools) and feed incidents back into test cases.

Question 14 | Testers Not Involved Early in SDLC

Scenario

Testers join only late in development. Apply the role of the tester in SDLC to fix this and improve collaboration.

1. Tester role in each SDLC phase

Phase	What the tester should do
Requirements	Review for clarity, testability and completeness; ask "how will I verify this?"
Design	Review design, identify risks, begin test strategy and data needs
Development	Prepare test cases, support unit/integration testing, early build checks
Testing	Execute tests, log and verify defects
Release/maintenance	Smoke test, production feedback, regression

2. How early involvement impacts quality

The later a defect is found, the more it costs to fix. A requirement defect found in review may cost a few minutes; the same defect found after release can cost many times more, because design, code, tests and data must all change.

Phase where defect is found	Relative cost to fix (illustrative)
Requirements	1x
Design	5x
Coding	10x
System testing	20x
Production	50x to 100x

3. Practical example

Ambiguous requirement: "System should allow quick refunds"

A tester in the review asks: "What is quick? Which payment types? Is there a limit?" The team defines "within 24 hours, up to the order value".

Without the tester, developers guess; the defect shows up at system testing or in production and needs redesign of the refund module.

4. Changes in team collaboration

- Include testers in requirement and design reviews and sprint planning (shift-left).
- Definition of Ready and Definition of Done require test criteria.
- Daily stand-ups, shared defect board, pair testing with developers, and three-amigos (BA, developer, tester) sessions.
- Developers share early builds so testers can run smoke tests continuously.

Question 15 | Correct in Some Cases, Failing in Others: Coverage Gaps

Scenario

A system works for some inputs and fails for others because test coverage is incomplete. Find the gaps and improve test case design.

1. Identifying gaps

Coverage type	Question asked	Typical gap
Requirement coverage	Is every requirement tested?	Rules with no linked test case
Input coverage	Are all input classes tested?	Only valid, typical data used
Boundary coverage	Are edges tested?	Min, max, just-outside values missing
Code coverage	Which statements/branches ran?	Error-handling branches never executed
Scenario coverage	Are combinations/paths tested?	Rare combinations skipped

2. How proper coverage ensures reliability

Reliability is the probability that the software works correctly over time and use. When tests cover all input classes, boundaries and paths, behaviour is verified across the range the users will actually exercise, so failures in "other cases" are found before release.

3. Improving test case design: before and after

Example: discount rule "10% off for orders from Rs. 1,000 to Rs. 4,999; 20% for Rs. 5,000 and above"

Before (gaps): tests for 2,000 and 6,000 only.

After: partitions (below 1,000, 1,000 to 4,999, 5,000 and above, invalid) plus boundaries 999, 1000, 4999, 5000, 0 and -1. A defect at exactly 5,000 (treated as 10%) is now caught.

4. Techniques for better validation

- Equivalence partitioning, boundary value analysis and decision tables for input and rule coverage.
- Requirement traceability matrix to show untested requirements.
- Statement and branch coverage tools (JaCoCo, Coverage.py) to expose unexecuted code.
- Pairwise testing for combinations; reviews and inspections of test cases.

Question 16 | Logistics: Delay Due to Poor Defect Reporting

Scenario

Defect identification is delayed because of poor reporting. Improve defect reporting using testing concepts.

1. How clear definitions help tracking

- Using the same meaning for **error**, **defect** and **failure** means everyone knows what is being logged: the observed failure and the suspected defect.
- Clear classes (severity, priority, status) let the team sort and track work without argument.
- A shared vocabulary reduces back-and-forth and prevents duplicates.

2. Poor report versus structured report

Poor report	Structured report
"Tracking page is slow and wrong."	ID: LOG-482 Title: Shipment status shows "Delivered" while the package is in transit Module: Tracking Steps: 1. Log in as customer. 2. Search AWB 784512. 3. Open status tab. Expected: "In transit". Actual: "Delivered". Environment: Chrome, build 4.2, UAT Severity: High. Priority: P1. Attachment: screenshot, API response.

3. How structured reports improve communication

Reader	Benefit
Developer	Can reproduce immediately and starts fixing without questions
Test lead	Can trace defect to test case and measure quality
Project manager	Sees severity and status to plan the release
Business/customer	Understands the impact in plain terms

4. Improvements in reporting standards

1. Adopt one defect template (IEEE 1044 style) with mandatory fields.
2. Define severity and priority scales with examples (for example, wrong delivery status = High).
3. Log defects immediately with evidence; check for duplicates first.
4. Set service levels: critical defects triaged within 4 hours.
5. Run daily triage and review report quality in retrospectives.

Question 17 | Inconsistent Testing Across Modules

Scenario

A team cannot keep testing consistent across modules. Apply process-based testing and show how standardisation improves quality.

1. Process-based approach



Every module follows the same documented process and templates. Differences between modules come from risk, not from personal habits of testers.

2. How standardised processes improve quality

Standard element	Quality effect
Common test plan and test case template	Same depth and format for all modules
Entry and exit criteria	No module is released with weak testing
Defined severity levels and defect lifecycle	Comparable defect data across modules
Review of test cases	Missed scenarios found before execution
Metrics (coverage, pass rate, defect density)	Objective comparison of modules

3. Example

Three modules: Orders, Inventory, Billing

Before: Orders tested with documented cases, Inventory by exploring, Billing only at the end. Billing defects reach production.

After: all three follow the same template, exit criteria (100% critical cases executed, no open critical defects, 90% requirement coverage) and a common regression suite. Defect density becomes comparable, and weak spots are visible.

4. Methods to ensure consistency

- Process checklists and templates; a test strategy document at project level.
- Process audits and peer reviews; a test lead owns compliance.
- Shared test management tool and automation framework with common naming rules.
- Training and onboarding; process improvement using retrospective data (for example, TMMi or ISO/IEC 29119 guidance).

Question 18 | Post-Deployment Failure from Untested Edge Cases

Scenario

The system fails after deployment because edge cases were untested. Explain why they were missed, how boundary testing helps, and how to include edge cases.

1. Why edge cases were missed

Cause	Explanation
Testing only "happy paths"	Tests used typical, valid data
Exhaustive testing is impossible	No technique used to choose the most defect-prone inputs
Weak requirements	Limits and exceptions were not stated, so no tests were designed
Time pressure	Negative and boundary tests were dropped
Pesticide paradox	Same tests reused and stopped finding new defects

2. How boundary testing helps

Defects cluster at the edges of input ranges, often because of mistakes like using `<` instead of `<=` in code. Boundary value analysis tests the values at and next to each boundary.

3. Worked example: age field accepts 18 to 60

Value	17	18	19	59	60	61
Expected	Reject	Accept	Accept	Accept	Accept	Reject
Actual (defect: age < 60)	Reject	Accept	Accept	Accept	Reject (fail)	Reject

Only the boundary test at 60 exposes the defect; typical values (25, 40) would pass.

4. Including edge cases in testing

Category	Examples
Numeric	Zero, negative, maximum, overflow, decimals
Text	Empty, one character, maximum length, special characters, Unicode
Date/time	Leap day, month end, time zone change, 31 December
Collections and files	Empty list, one item, maximum size, duplicate entries
System	Network loss, full disk, concurrent users, session timeout

5. Improvements in test design

- Make BVA, equivalence partitioning and negative testing mandatory in test design review.
- Add edge-case checklists to requirement reviews and ask for limits in every requirement.
- Use exploratory sessions and refresh test data each cycle; turn production incidents into new test cases.

Question 19 | No Proper Documentation of Defects

Scenario

The company lacks defect documentation, making tracking hard. Apply defect repository concepts to fix this.

1. How documentation improves defect management

- Defects are not lost or duplicated; each has an owner, status and history.
- Developers can reproduce and fix quickly; testers can verify against the same record.
- Management can see status, risk and trends and make release decisions.

2. Structure of a defect repository entry

Field	Purpose
Defect ID, title	Unique reference and short summary
Module/component, version	Where it was found
Steps, expected, actual result	Reproduction and comparison
Severity, priority	Impact and urgency
Type, origin phase	Classification for analysis
Status, owner, dates	Tracking through the lifecycle
Root cause, fix, retest result	Prevention and verification
Attachments, linked test case/requirement	Evidence and traceability

3. Defect lifecycle recorded in the repository



Defects may also move to Reopened (retest fails), Deferred or Rejected (not a defect), each with a reason recorded.

4. How a structured repository supports analysis

Report	Question answered
Defects by module	Where are the defect clusters?
Defects by severity and age	What is risky for release? What is stuck?
Defects by origin phase	Which phase needs better reviews?
Open versus closed trend	Is quality converging?
Reopen rate	Are fixes reliable?

5. Improvements in documentation practices

1. Adopt a tool (Jira, Bugzilla, Azure DevOps) and a single template.
2. Make key fields mandatory; use dropdowns for type, severity and module.
3. Document at the moment of discovery with screenshots and logs.

-
4. Review documentation quality in triage; archive and mine closed defects for lessons.

Question 20 | Banking: Repeated Transaction Failures

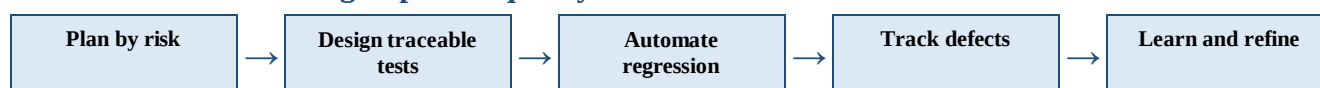
Scenario

Transaction processing fails repeatedly due to improper testing. Redesign the approach using testing principles and improve defect detection.

1. Redesign: problem, principle, solution

Problem observed	Principle	Redesigned practice
Same failures recur	Pesticide paradox	Regression suite refreshed with every fixed defect and new scenario
Failures in few services (payment posting, settlement)	Defect clustering	Risk-based effort on high-defect transaction modules
Defects found late	Early testing	Test review at requirements and design; automated checks per build
Only normal cases tested	Exhaustive testing is impossible	Select inputs by BVA, EP, decision tables and state models
Passing tests assumed "no bugs"	Testing shows presence of defects	Report residual risk and coverage, not a promise of zero defects
Tests do not reflect real use	Context dependence	Include security, concurrency, audit and regulatory scenarios

2. How structured testing improves quality



3. Illustration

Transfer Rs. 10,000 between accounts

Cases designed: sufficient balance, exact balance, insufficient balance by Rs. 1, closed beneficiary account, duplicate click, server timeout after debit, two simultaneous transfers from the same account.

Defect found: the timeout case debits the sender but never credits the receiver (no rollback). Severity critical; fix is transaction atomicity (commit/rollback) with a reconciliation test.

4. Techniques to enhance defect detection

Technique	Contribution
Boundary value and equivalence partitioning	Limits, balances and amounts
State transition testing	Transaction states: initiated, authorised, posted, reversed
Decision tables	Rules combining account type, limit, balance and status
Concurrency and performance testing	Simultaneous transactions and peak load

Technique	Contribution
Security testing	Authentication, tampering, injection
Reviews, defect analysis, automation	Early defect removal and stable regression

Conclusion: applying testing principles with structured design, risk-based effort, automation and defect analysis turns repeated failures into early, controlled findings.